INSTRUCTOR MANUAL

CSE 4571 — Practical Robotics Project with Arduino

Seventh Semester B.Tech (2L – 4P) | Hardware-Integrated Simulation-Based Laboratory

Practical Arduino Robotics — Lukas Kaul

SESSION-3

Servo Motor Control for Position-Based Robotic Actuation

Learning Domain Distribution — Cognitive: 45% | Psychomotor: 40% | Affective: 15%

*Note: This is a **Hardware-Integrated Simulation-Based Laboratory** course, where simulation activities are supported by actual hardware components. **Physical components, breadboards, and hardware kits are required** for laboratory activities, along with the prescribed simulation tools.*



SIKSHA 'O' ANUSANDHAN

1. Session Information

Field	Details
Course Code / Title	CSE 4571 — Practical Robotics Project with Arduino
Semester	Seventh Semester B.Tech (CSE)
Course Format	2L – 4P (Practical / Laboratory, Hardware-Integrated and Simulation-Based)
Session	3
Duration	6 Hours (360 minutes)
Software and Hardware Used	Tinkercad/ Wokwi (Online Circuit Simulation Platform) and physical components
Topic	Servo Motor Control for Position-Based Robotic Actuation
Textbook Reference	Practical Arduino Robotics — Lukas Kaul

*Hardware components, apparatus, component kits, and breadboards are integrated where required, while simulation-based activities are performed on-screen using **Tinkercad/Wokwi**.*

2. Learning Objectives

By the end of this session, students will be able to:

Objectives	Statements
O1	Explain the fundamental concept of servo motor operation and its role in position-based robotic actuation.
O2	Differentiate between a servo motor as a position-controlled actuator and conventional motors used for continuous rotational motion, and identify the major terminals and functions of a micro-servo.
O3	Explain the role of a potentiometer as a position command and describe how its analog voltage is converted by the Arduino ADC into a digital value.
O4	Identify and correctly wire the Arduino UNO, potentiometer and micro-servo in a simulation or hardware setup, including signal, power and common-ground connections.
O5	Implement and observe Arduino-based servo position control by mapping ADC values to servo angles, applying a restricted operating range, and relating the control method to practical robotic applications.

1. Course Outcomes (CO1–CO6)

CO	Course Outcome Statement	Learning Domain	Taxonomy Level
CO1	Understand the fundamentals of Arduino hardware and software	Cognitive (L) + Psychomotor (P) + Affective (A)	L2 – Understand+ P1: Perception+ A1: Receiving
CO2	Interface various sensors and actuators with Arduino	Cognitive (L) + Psychomotor (P) + Affective (A)	L3 – Apply + P3 – Guided Response+ A2: Responding
CO3	Apply programming logic to control robotic systems.	Cognitive (L) + Psychomotor (P) + Affective (A)	L3 – Apply + P4 – Mechanism + A3: Valuing
CO4	Design and build basic to intermediate level robotics projects.	Cognitive (L) + Psychomotor (P) + Affective (A)	L5 – Evaluate + P5 – Complex Overt Response+ A4: Organization
CO5	Demonstrate problem-solving and debugging skills in robotics.	Cognitive (L) + Psychomotor (P) + Affective (A)	L4 – Analyze + P6 – Adaptation + A4: Organization
CO6	Collaborate in teams to plan, execute, and present robotics projects.	Cognitive (L) + Psychomotor (P) + Affective (A)	L6 – Create + P7 – Origination + A5 – Characterization

L: Cognitive (L1: Remembering; L2: Understanding; L3: Applying; L4: Analyzing; L5: Evaluating; L6: Creating)

P: Psychomotor (P1: Perception; P2: Set; P3: Guided Response; P4: Mechanism; P5: Complex Overt Response; P6: Adaptation; P7: Origination)

A: Affective (A1: Receiving; A2: Responding; A3: Valuing; A4: Organization; A5: Characterization)

The course CSE 4571 - Practical Robotics Project with Arduino primarily reflects the following Sustainable Development Goals (SDGs):

UN Sustainable Development Goals (SDGs)

SDG	Goal	Course Relevance
SDG 4	Quality Education	Develops practical skills in Arduino programming, servo motor control, potentiometer interfacing, ADC-to-angle mapping, circuit implementation, simulation, observation, and troubleshooting.
SDG 7	Affordable and Clean Energy	Introduces controlled and efficient operation of electronic actuators by applying appropriate control signals and safe operating ranges, supporting awareness of responsible energy use in robotic systems.
SDG 9	Industry, Innovation and Infrastructure	Promotes innovation in robotic system development through Arduino-based position control, simulation/hardware implementation, actuator interfacing, and applications in robotic arms, grippers, pan-tilt systems, and steering mechanisms.
SDG 12	Responsible Consumption and Production	Encourages responsible engineering practices through simulation-based testing, systematic troubleshooting, safe actuator operation, and software-based range restriction to avoid unnecessary stress or damage to components.

CO – PO Mapping (Focus: CO1-CO6)

CO	PO Mapped	Strength
CO1	PO1, PO2, PO3	High
CO2	PO1, PO2, PO3, PO4, PO5, PO11	High
CO3	PO1, PO2, PO3, PO4, PO5, PO11	High
CO4	PO1, PO2,PO3, PO4, PO5, PO6, PO7, PO8, PO9, PO11	High
CO5	PO1, PO2,PO3, PO4, PO5, PO6, PO7, PO8, PO9,PO10, PO11	High
CO6	PO1, PO2,PO3, PO4, PO5, PO6, PO7, PO8, PO9,PO10, PO11	High

2. Learning Domain Distribution (3H Model)

The 360-minute session strictly maintains the prescribed course-level domain distribution of Cognitive 45%, Psychomotor 40%, and Affective 15%.

Domain	Weightage	Time (approx.)	Focus
Cognitive (Head)	45%	162 minutes	Understanding servo motor operation, position-based robotic actuation, potentiometer as a position command, ADC, ADC-to-angle mapping, Arduino programming, range restriction, and analysis of servo position control.
Psychomotor (Hand)	40%	144 minutes	Identifying and wiring Arduino, potentiometer and servo; running the Arduino program; operating the potentiometer; observing servo movement; implementing 20°–160° range restriction; simulation/hardware testing and troubleshooting.
Affective (Heart)	15%	54 minutes	Developing safety awareness, careful wiring, systematic observation, responsible handling, teamwork, recognition of mechanical limits, and safe actuator operation.

3. Instructor Preparation

Before opening **Tinkercad/Wokwi**, the instructor should explain the basic concepts of servo motor control. Introduce the role of the **Arduino** as the controller for position-based robotic actuation. Explain the **potentiometer** as a manual input for specifying the desired servo position.

Briefly explain **analog voltage, ADC, and ADC-to-angle mapping** in simple terms. Demonstrate the relationship between potentiometer position, servo command, and angular movement. Physical components may be shown as references, while activities are performed through **simulation and integrated hardware**, where applicable.

Servo Motor and Position-Based Robotic Actuation

A servo motor is a compact actuator designed to move its output shaft toward a commanded angular position. It is widely used in robotic mechanisms because the actuator can be commanded to a particular position rather than simply being switched on for continuous rotation. In a position-based robotic

actuation system, the desired position is treated as the command variable. The controller receives this command, processes it and generates a corresponding actuator command. The servo then produces mechanical movement toward the requested position. The experiment therefore demonstrates the fundamental chain of command generation, signal processing and mechanical actuation. The servo contains a small motor, gearing and internal control electronics. The Arduino does not directly drive the motor windings; instead, it sends a control signal representing the desired position. This makes the servo useful for simple robotic joints, grippers, pan-tilt systems and other mechanisms where controlled angular positioning is required.

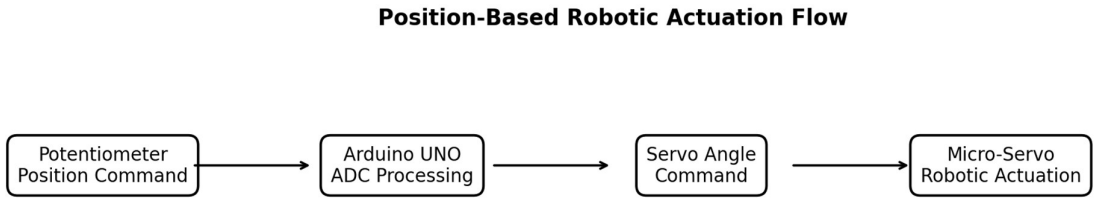


Figure 1. Functional flow of potentiometer-based position control for robotic actuation.

Theory: Potentiometer as a Position Command

A potentiometer is a three-terminal variable resistor. When its outer terminals are connected to supply and ground, the center terminal produces a variable voltage as the knob is rotated. This voltage represents the user's desired actuator position. The Arduino reads this voltage through analog input A0. The input is converted by the ADC into a digital number. The program then interprets this number as a position command and converts it into a servo angle. Thus, the potentiometer is an input device rather than a power source for the servo.

Circuit Identification and Wiring

Students should identify the potentiometer VCC, GND and SIG terminals and the servo VCC, GND and signal terminals before wiring. The potentiometer signal is connected to A0, while the servo signal is connected to D9. The grounds are common.

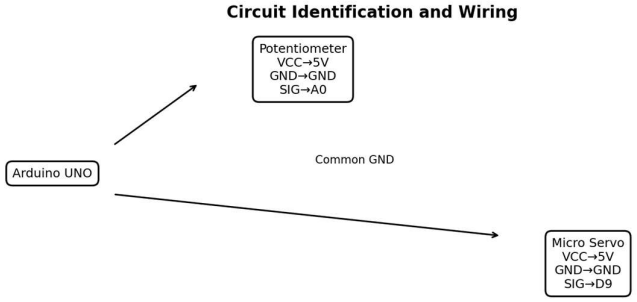


Figure 2. Circuit identification and wiring for potentiometer-based servo position control.

Theory: ADC and Position-Control Concept

The Arduino UNO uses a 10-bit ADC for the analog input used in this experiment. The potentiometer voltage is represented by an ADC value from 0 to 1023. The program maps this numerical range to a servo-angle range. In the initial version, 0–1023 is mapped to 0°–180°. The complete position-control concept is: potentiometer position → analog voltage → ADC value → mapped angle → servo actuator movement. Students should be able to explain this flow before proceeding to programming.

Objectives

- Understand the working principle of a servo motor and its role as a position-controlled actuator in robotic systems.
- Understand position-based robotic actuation, in which an input command is converted into a desired mechanical position.
- Identify the Arduino UNO, potentiometer and servo motor terminals and understand their functions.
- Interface the potentiometer with the Arduino analog input and the servo with a digital control pin.
- Understand how the potentiometer generates a variable analog voltage representing the desired actuator position.
- Understand analog-to-digital conversion (ADC) and convert the ADC value into a corresponding servo-angle command.
- Develop the Arduino program required to read the position command and control the servo motor.
- Observe and analyze the relationship between potentiometer position, ADC value and servo angular position.
- Apply software-based angular limits for safe and controlled operation of a servo-driven robotic mechanism.
- Relate servo position control to robotic arms, grippers, pan-tilt mechanisms, steering systems and prototype testing.

Components Required (Tinkercad/Wokwi and Hardware)

Component	Quantity
Arduino UNO	1
Micro Servo	1
Potentiometer	1
Breadboard	1
Jumper wires	As required
USB cable / power source	1

Arduino Program – Basic Position Control

```
#include <Servo.h>
Servo servo;
const int potPin = A0;
const int servoPin = 9;
void setup() {
  servo.attach(servoPin);
  Serial.begin(9600);
}
void loop() {
  int adcValue = analogRead(potPin);
  int angle = map(adcValue, 0, 1023, 0, 180);
  servo.write(angle);
  Serial.print("ADC: ");
  Serial.print(adcValue);
  Serial.print("  Angle: ");
  Serial.println(angle);
  delay(15);
}
```

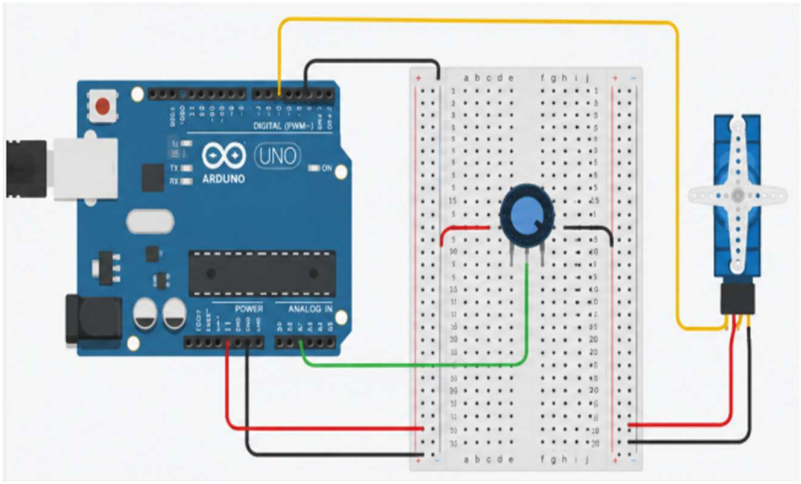


Figure 3. Tinkercad/Wokwi circuit setup for potentiometer-based servo motor position control using Arduino UNO.

Figure 4. Hardware setup for potentiometer-based servo motor position control using Arduino UNO.

4. Software & Resources Required

Resource	Purpose
Tinkercad (www.tinkercad.com)	Primary simulation platform for circuit design, Arduino programming, and servo motor position-control activities.
Wokwi (wokwi.com)	Alternative simulation platform for Arduino-based circuit design, programming, and testing of servo motor position control.
Web Browser (Chrome/Edge, updated)	Tinkercad and Wokwi run directly in the browser; no software installation is required.
Projector / Screen Share	Enables live instructor demonstrations of circuit wiring, programming, simulation, and servo operation.
Observation Sheet	Used by students to record potentiometer positions, ADC values, servo angles, observations, and experimental outcomes.

*This experiment follows a **Hardware-Integrated and Simulation-Based** approach. Physical components, kits, and Arduino-based hardware are used for hands-on implementation and verification, while **Tinkercad /Wokwi** are used for circuit design, programming, simulation, and servo motor position-control testing.*

5. Student Preparation (Pre-Lab)

- Create a free **Tinkercad/Wokwi** account prior to the session or join the instructor's Tinkercad/Wokwi Classroom using the provided class code.
- Review the pre-lab material on **servo motors, potentiometers, Arduino UNO, ADC, and position-based robotic actuation** before the session.
- Bring a notebook to record circuit connections, ADC values, servo angles, observations, and experimental results.

6. Minute-wise Instructor & Student Activity Plan

Time	Instructor Activity	Student Activity	Domain
0–20 min	Introduce the experiment, aim, objectives, laboratory/simulation practices, safety requirements, and applications of servo position control in robotics.	Listen to the introduction, recognize the importance of safe laboratory/simulation practices, and show readiness to participate in the activity.	Affective (Heart)
20–60 min	Explain the working principle of a micro-servo as a position-controlled actuator and discuss position-based robotic actuation and practical robotic applications.	Understand servo operation and explain its role in robotic arms, grippers, pan-tilt systems and steering mechanisms.	Cognitive (Head)
60–95 min	Explain the potentiometer as a manual position command, analog voltage generation, Arduino analog input, ADC operation, and the 0–1023 ADC range.	Relate potentiometer position to analog voltage and ADC value and explain how the input represents the desired servo position.	Cognitive (Head)
95–135 min	Demonstrate Arduino UNO, potentiometer, micro-servo and breadboard identification and show VCC, GND, A0 and D9 connections in Tinkercad/Wokwi or hardware.	Identify the components and terminals, place/arrange the components, build the circuit, and verify VCC, GND, A0, D9 and common-ground connections.	Psychomotor (Hand)
135–180 min	Explain ADC-to-angle mapping from 0–1023 to 0°–180° and explain the Arduino program using Servo library, Servo.attach(9), analogRead(A0), map(), Servo.write() and Serial Monitor.	Interpret the program statements, calculate/relate ADC values to servo angles, and explain the complete potentiometer → ADC → mapped angle → servo movement sequence.	Cognitive (Head)
180–220 min	Demonstrate program entry/upload and guide students in running the basic servo position-control program in simulation or hardware.	Enter/upload the Arduino program, start the simulation or hardware test, operate the potentiometer, and verify that the servo responds correctly.	Psychomotor (Hand)
220–260 min	Guide systematic experimentation using minimum, intermediate and maximum potentiometer positions and demonstrate use of the Serial Monitor.	Operate the potentiometer through different positions and record at least five ADC values, expected angles and observed servo angles.	Psychomotor (Hand)

260–302 min	Explain servo range restriction from 0°–180° to 20°–160°, the need for mechanical limits, expected restricted angles, deviations, and troubleshooting logic.	Analyze the effect of range restriction, compare expected and observed positions, and identify possible causes of errors involving wiring, code, power, servo orientation or mechanical loading.	Cognitive (Head)
302–326 min	Guide students in modifying the mapping to 20°–160° and testing the restricted operating range in simulation or hardware.	Modify the program, run the system, verify minimum/intermediate/maximum restricted positions, and correct simple wiring or program errors.	Psychomotor (Hand)
326–360 min	Discuss safe actuator operation, responsible handling, teamwork and practical robotic applications; conduct review/Q&A and provide post-lab instructions.	Participate in discussion and Q&A, demonstrate safety awareness and teamwork, value simulation/hardware-integrated learning, summarize the experiment, and note post-lab work.	Affective (Heart)

7. Instructor Demonstration Steps (Simulation and Hardware Implementation)

- **Step 1:** Sign in to **Tinkercad** → create a new Arduino circuit/project.
- **Step 2:** Add an **Arduino UNO, potentiometer, servo motor, breadboard, and jumper wires** to the workspace/Breadboard.
- **Step 3:** Connect the potentiometer to **5V, GND, and A0**, and connect the servo to **5V, GND, and digital pin D9** with a common ground.
- **Step 4:** Enter the Arduino program to read the potentiometer value using analogRead(A0), map the ADC range **0–1023 to 0°–180°**, and control the servo using Servo.write().
- **Step 5:** Start the simulation and rotate the potentiometer; demonstrate how the **ADC value and servo angular position change** with the input command.
- **Step 6:** Open the Serial Monitor and demonstrate the displayed **ADC value and corresponding servo angle**.
- **Step 7:** Change the program to restrict the servo movement to **20°–160°** and demonstrate the restricted operating range.
- **Step 8:** Stop the simulation, verify the circuit and program, save the project, and rename it as **Servo_Position_Control_[InstructorName]**.
- **Step 9: Repeat step2 to step 8 procedure using hardware components for physical implementation of the experiment**

8. Guided / Probing Questions

- Modify the map() function to restrict servo movement from 0°–180° to 20°–160°.
- Why is the potentiometer signal connected to Arduino analog pin A0?
- Write the Arduino code required to display both the ADC value and mapped servo angle on the Serial Monitor.
- What does the map() function do in the servo position-control program?
- Modify the program so that the servo operates only between 30° and 150°, while the potentiometer still uses the full ADC range of 0–1023.

- What is the purpose of Servo.attach(9)?
- The servo is connected to D10, but the program uses servo.attach(9). What program modification is required, and what happens if it is not corrected?
- The potentiometer value changes in the Serial Monitor, but the servo does not move. Identify possible hardware connection faults.
- For an ADC value near 512, what angle is approximately commanded in the 0°–180° mapping?
- How would you modify the program to restrict the servo to 20°–160°?
- Why is range restriction important when a servo drives a robotic joint?
- Explain why the Arduino, potentiometer, and servo motor must share a common ground.
- The servo vibrates continuously instead of reaching a stable position. How would you troubleshoot the power supply, wiring, and mechanical loading?
- How would you modify the hardware circuit to control two servo motors simultaneously using one Arduino?
- What problems can occur if a servo motor is powered directly from the Arduino's 5V pin, and how can they be avoided?
- Suggest one practical robotic application in which a potentiometer can be used as a manual position command.

9. Hands-on Activity (Student Task)

Task: Build and simulate a potentiometer-based servo motor position-control circuit using **Tinkercad/ Wokwi and Hardware.**

- Log in and create a project named Servo_Position_Control_[YourName]_[RollNo].
- Place an Arduino UNO, potentiometer, servo motor, breadboard, and jumper wires, and complete the required connections.
- Run the Arduino program to map the potentiometer ADC value (0–1023) to the servo angle (0°–180°) and observe the servo movement.
- Implement the hardware with precaution while handling hardware circuit connection.
- Record at least five observations of potentiometer position, ADC value, and corresponding servo angle, then verify the restricted 20°–160° operating range.
- Save the project and keep the Tinkercad/Wokwi project link ready for instructor review.
- Take the snapshot/video of working hardware for instructor review.

10. Discussion Points

- Servo motor applications in robotics - Discuss the use of position-controlled servo motors in robotic arms, grippers, pan-tilt mechanisms, and steering systems.
- Position-based control concept - Discuss how the potentiometer provides a position command, which the Arduino converts from an ADC value into the corresponding servo angle.
- Simulation, hardware integration, and safe operation - Discuss the use of Tinkercad/ Wokwi for simulation and testing, along with proper wiring, angular limits, systematic observations, precaution while handling hardware circuit connection and organized project management.

11. Reflection Questions (End of Session)

- What is one new concept you learned about servo motor position control today?
- Which part of the Tinkercad/Wokwi simulation did you find most intuitive, and which part was confusing?

- How confident do you feel about explaining the relationship between potentiometer position, ADC value, and servo angle (rate 1–5), and why?

12. Home Assignment

- Modify the Tinkercad/Wokwi servo control circuit and record five sets of potentiometer position, ADC value, and corresponding servo angle.
- Modify the Arduino program to restrict the servo movement to 20°-160° and verify the operation through simulation.
- Write a short note explaining one robotic application of position-based servo control, such as a robotic arm, gripper, or pan-tilt mechanism.

13. Assessment Rubrics

Rating Scale: 4 = Excellent, 3 = Good, 2 = Satisfactory, 1 = Needs Improvement (aligned to course domain weightage: Cognitive 45% / Psychomotor 40% / Affective 15%)

Criterion	Domain	4 – Excellent	3 – Good	2 – Satisfactory	1 – Needs Improvement
Component Identification & Circuit Construction	Cognitive / Psychomotor	Correctly identifies all components and independently builds the circuit with accurate connections.	Identifies most components and builds the circuit with minor errors.	Identifies some components and requires assistance with circuit construction.	Unable to identify components or construct the circuit correctly.
Servo Position Control & Experimentation	Cognitive / Psychomotor	Successfully controls servo position, records ADC values and angles accurately, and clearly explains the relationship.	Successfully performs position control and records most observations correctly.	Performs the experiment with assistance and records limited observations.	Unable to perform or record the position-control experiment.
Programming & Simulation	Cognitive / Psychomotor	Correctly implements the Arduino program, ADC-to-angle mapping, and simulation; independently troubleshoots errors.	Runs the program and simulation with minor errors or assistance.	Requires considerable assistance with programming or simulation.	Unable to run or modify the program or simulation.
Laboratory Engagement & Teamwork	Affective	Participates actively, follows laboratory practices, maintains an organized workspace, and supports peers.	Participates regularly and cooperates well with peers.	Participates when prompted and shows limited teamwork.	Remains passive, disengaged, or does not cooperate with peers.

14. Student Observation Sheet (Template)

Sl. No.	Potentiometer Position	ADC Value	Expected Angle (°)	Observed Angle (°)	Deviation (°)
1					
2					
3					

15.Expected Learning Outcome

At the end of the session, a student should be able to independently open Tinkercad/Wokwi, construct and simulate a potentiometer-based servo motor control circuit using Arduino UNO, correctly program the ADC-to-angle mapping, operate the servo through different angular positions, record the corresponding ADC values and servo angles, and explain in their own words how position-based servo control is used in robotic actuation.

16. Instructor Tips

- Keep the initial explanation concise and focused on servo motor operation, potentiometer-based position control, Arduino, ADC, and servo-angle mapping.
- Closely monitor students during the hands-on activity, particularly while making Arduino–potentiometer–servo connections and troubleshooting wiring or programming errors.
- Encourage peer-assisted learning and emphasize the relationship between potentiometer position → ADC value → mapped angle → servo movement.
- Maintain the Hardware-Integrated and Simulation-Based approach, using physical hardware for hands-on verification and Tinkercad/Wokwi for circuit design, programming, simulation, and testing.

CSE 4571 — Practical Robotics Project with Arduino

Basic Questions

No.	Question	Learning Domain	Taxonomy Level
1	What is a servo motor?	Cognitive	Bloom's Taxonomy – Remember
2	Name any two applications of servo motors in robotics.	Cognitive	Bloom's Taxonomy – Remember
3	What is meant by position-based robotic actuation?	Cognitive	Bloom's Taxonomy – Understand
4	What is the role of an Arduino UNO in servo motor control?	Cognitive	Bloom's Taxonomy – Understand
5	What is a potentiometer?	Cognitive	Bloom's Taxonomy – Remember
6	Why is a potentiometer used as an input in this experiment?	Cognitive	Bloom's Taxonomy – Understand
7	What is an analog input, and which Arduino pin reads the potentiometer?	Cognitive	Bloom's Taxonomy – Remember
8	What is ADC, and what is the ADC range of the Arduino UNO?	Cognitive	Bloom's Taxonomy – Remember
9	What is the purpose of the servo signal connection?	Cognitive	Bloom's Taxonomy – Understand
10	Which Arduino pin is used for the servo signal in this experiment?	Cognitive	Bloom's Taxonomy – Remember
11	What is the relationship between the potentiometer position and servo angle?	Cognitive	Bloom's Taxonomy – Understand
12	What happens to the servo angle when the potentiometer is rotated from minimum to maximum?	Cognitive	Bloom's Taxonomy – Analyze
13	Why is the potentiometer connected to an analog input rather than a digital input?	Cognitive	Bloom's Taxonomy – Understand
14	How is an ADC value mapped to a servo angle?	Cognitive	Bloom's Taxonomy – Understand
15	Calculate the approximate servo angle for an ADC value of 512 when the range is 0°–180°.	Cognitive	Bloom's Taxonomy – Apply

16	What servo angle corresponds approximately to the minimum ADC value?	Cognitive	Bloom's Taxonomy – Apply
17	What servo angle corresponds approximately to the maximum ADC value?	Cognitive	Bloom's Taxonomy – Apply
18	Why is simulation useful before implementing the servo-control circuit on physical hardware?	Cognitive	Bloom's Taxonomy – Understand
19	Can you locate the Circuits option in Tinkercad or create a new Arduino project in Wokwi?	Psychomotor	Simpson's Taxonomy – Perception
20	Can you identify the Arduino UNO, potentiometer, servo motor, workspace, and simulation controls?	Psychomotor	Simpson's Taxonomy – Perception
21	Can you search for and place the Arduino UNO, potentiometer, servo motor, and breadboard in the workspace?	Psychomotor	Simpson's Taxonomy – Set
22	Can you arrange the Arduino, potentiometer, and servo motor correctly in the workspace?	Psychomotor	Simpson's Taxonomy – Guided Response
23	Can you identify the VCC, GND, A0, and D9 connections required for the experiment?	Psychomotor	Simpson's Taxonomy – Perception
24	Can you connect the potentiometer to 5V, GND, and A0?	Psychomotor	Simpson's Taxonomy – Guided Response
25	Can you connect the servo motor to 5V, GND, and D9 with the correct connections?	Psychomotor	Simpson's Taxonomy – Guided Response
26	Can you start the simulation and verify that the servo responds to potentiometer movement?	Psychomotor	Simpson's Taxonomy – Mechanism
27	Can you change the potentiometer position and record the corresponding ADC value and servo angle?	Psychomotor	Simpson's Taxonomy – Mechanism
28	Are you willing to follow proper simulation-laboratory practices while using Tinkercad/Wokwi?	Affective	Krathwohl's Taxonomy – Receiving
29	How actively did you participate and support your peers during the servo-control activity?	Affective	Krathwohl's Taxonomy – Responding
30	Do you value simulation and hardware-integrated servo control as an effective method for learning robotic actuation? Why?	Affective	Krathwohl's Taxonomy – Valuing